

Celebal Technologies Internship – Week 6 Assignment

Apache Spark Architecture & Data Processing

Name: Abhishek Raj

Internship: Celebal Technologies Data Engineering Internship

Week: 6

Objective

The objective of this assignment is to understand Apache Spark architecture and implement efficient data processing using PySpark. The assignment focuses on Spark architecture components, Lazy Evaluation, Directed Acyclic Graph (DAG), schema handling, DataFrame transformations and actions, filtering, data type conversion, Predicate Pushdown, and working with CSV and Parquet file formats. It also demonstrates building scalable data processing pipelines following Spark best practices.

Dataset Description

A custom **Retail Sales** dataset was created specifically for this assignment using Python libraries including Pandas, NumPy, and Faker. The dataset contains realistic retail transaction records with multiple product categories, regions, payment methods, priorities, and customer details. Missing values and duplicate records were intentionally introduced to simulate real-world data processing scenarios and demonstrate Spark's data transformation capabilities.

Dataset Columns

- user_id
- product_id
- transaction_id
- category
- product_name
- old_name

- price
 - base_price
 - quantity
 - amount
 - status
 - region
 - priority
 - order_date
 - payment_method
 - customer_name
 - city
 - state
-

Software Requirements

- Python
 - Apache Spark (PySpark)
 - Java
 - Jupyter Notebook / VS Code
 - Pandas
 - NumPy
 - Faker
-

Assignment Questions

Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

Answer

Apache Spark follows a distributed architecture where the **Driver**, **Cluster Manager**, and **Executors** work together to process data efficiently.

Driver

- Acts as the main controller of the Spark application.
- Creates the `SparkSession`.
- Converts user code into jobs, stages, and tasks.
- Schedules tasks to executors.
- Collects and returns the final results.

Cluster Manager

- Allocates resources across the cluster.
- Launches executors on worker nodes.
- Manages CPU cores and memory for Spark applications.
- Examples include Standalone, YARN, Kubernetes, and Mesos.

Executor

- Runs on worker nodes.
- Executes tasks assigned by the Driver.
- Performs data processing and computations.
- Stores cached data when required.
- Sends the processed results back to the Driver.

Summary

Component	Role
Driver	Controls the application, schedules tasks, and collects results.
Cluster Manager	Manages cluster resources and launches executors.
Executor	Executes tasks, processes data, and returns results to the Driver.

Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

Answer

Spark uses **Lazy Evaluation**, which means transformations are not executed immediately. Instead, Spark records all transformations in a **Directed Acyclic Graph (DAG)** and waits until an action such as `show()`, `count()`, or `collect()` is called.

This strategy improves performance by:

- Eliminating unnecessary computations.
- Optimizing the execution plan before running the job.
- Combining multiple transformations into fewer execution stages.
- Reducing disk I/O and network communication.
- Minimizing data shuffling between executors.

As a result, Spark executes only the required operations, making large-scale data processing faster and more efficient.

Q3. Read a CSV file with header and inferSchema enabled.

Explanation

The retail sales dataset was loaded into a Spark DataFrame using the `spark.read.csv()` function with header detection and automatic schema inference enabled. This allows Spark to identify column names and infer appropriate data types automatically.

Code Screenshot

PySpark Code

```
df = spark.read \
    .option("header", True) \
    .option("inferSchema", True) \
    .csv("../dataset/week6_spark_dataset.csv")

df.show(5)
```

✓ 0.3s

Output Screenshot

user_id	product_id	transaction_id	category	product_name	old_name	price	base_price	quantity	amount	status	region	priority	order_date	payment_method	customer_name	c
4657	P28289	TXN100001	Furniture	Chair	Chair	37103.37	37103.37	4	148413.48	Completed	central	High	2025-08-27	UPI	Julie Miller	De
1434	P83563	TXN100002	Furniture	Table	Table	11709.78	11709.78	10	117097.8	Pending	central	Medium	2024-08-13	UPI	William Torres	Kolk
5552	P30379	TXN100003	Electronics	Smartphone	Smartphone	34937.16	34937.16	6	209622.96	Pending	East	High	2024-04-04	UPI	Sarah Byrd	Mum
8527	P80284	TXN100004	Groceries	Sugar	Sugar	13299.59	13299.59	1	13299.59	Completed	West	High	2025-07-19	Debit Card	Erica Stanley	Kolk
4733	P47930	TXN100005	Automotive	Car Cover	Car Cover	35258.13	35258.13	1	35258.13	Completed	South	High	2025-01-24	Debit Card	Elizabeth Nelson	Luck

only showing top 5 rows

Q4. Difference between CSV and Parquet.

Answer

CSV	Parquet
Row-based storage	Columnar storage
Plain text format	Binary compressed format
Larger file size	Smaller file size
Slower read performance	Faster read performance
No built-in schema	Stores schema information
Limited compression	Highly compressed

Why it matters

Parquet stores data column-wise, allowing Spark to read only the required columns instead of the entire dataset. This reduces disk I/O, memory usage, and improves query performance, making Parquet the preferred format for big data analytics.

Q5. Select product_id and price where category is Electronics.

Explanation

The DataFrame was filtered to include only records where the product category is **Electronics**. Only the **product_id** and **price** columns were selected from the filtered dataset.

Code Screenshot

```
PySpark Code

from pyspark.sql.functions import col

df.filter(col("category") == "Electronics") \
    .select("product_id", "price") \
    .show()

] ✓ 0.2s
```

Output Screenshot

```
+-----+-----+
|product_id|  price|
+-----+-----+
|   P30379|34937.16|
|   P27224|39451.66|
|   P18992|12254.09|
|   P78955|13341.84|
|   P77552|44300.94|
|   P44311|25002.29|
|   P16731|23782.05|
|   P14888|32649.71|
|   P27575|20692.61|
|   P80371|10368.65|
|   P84498|46702.83|
|   P15220| 3469.88|
|   P36895| 42465.4|
|   P58823|44874.64|
|   P45200|22043.05|
|   P88167|47428.32|
|   P59840|34518.51|
|   P45386|21101.49|
|   P78417|17821.44|
|   P39563|37209.13|
+-----+-----+
only showing top 20 rows
```

Q6. Rename old_name and cast price to Double.

Explanation

The **old_name** column was renamed to **new_name** using `withColumnRenamed()`. The **price** column was then converted from String to Double using the `cast()` function for numerical processing.

Code Screenshot

```
PySpark Code

from pyspark.sql.functions import col

df = df.withColumnRenamed("old_name", "new_name")

df = df.withColumn(
    "price",
    col("price").cast("double")
)

df.printSchema()

✓ 0.0s
```

Output Screenshot

```
root
|-- user_id: integer (nullable = true)
|-- product_id: string (nullable = true)
|-- transaction_id: string (nullable = true)
|-- category: string (nullable = true)
|-- product_name: string (nullable = true)
|-- new_name: string (nullable = true)
|-- price: double (nullable = true)
|-- base_price: double (nullable = true)
|-- quantity: integer (nullable = true)
|-- amount: double (nullable = true)
|-- status: string (nullable = true)
|-- region: string (nullable = true)
|-- priority: string (nullable = true)
|-- order_date: date (nullable = true)
|-- payment_method: string (nullable = true)
|-- customer_name: string (nullable = true)
|-- city: string (nullable = true)
|-- state: string (nullable = true)
```

Q7. Explain Spark Lineage Graph (DAG).

Answer

Spark maintains a **Lineage Graph (Directed Acyclic Graph - DAG)** that records every transformation performed on a dataset. Instead of replicating data multiple times, Spark uses this lineage information to recover lost data.

If a worker node or executor fails:

- Spark identifies the lost partitions.
- It traces the transformations stored in the DAG.
- Only the missing partitions are recomputed.
- The remaining completed partitions are reused.

This approach provides efficient fault tolerance while reducing storage overhead.

Q8. Filter Completed orders having amount greater than 1000.

Explanation

The DataFrame was filtered using multiple conditions to select only those records where the **status** is **Completed** and the **amount** is greater than **1000**.

Code Screenshot

PySpark Code

```
from pyspark.sql.functions import col

df.filter(
    (col("status") == "Completed") &
    (col("amount") > 1000)
).show()

✓ 0.2s
```

Output Screenshot

user_id	product_id	transaction_id	category	product_name	new_name	price	base_price	quantity	amount	status	region	priority	order_date	payment_method	customer_name
4657	P28289	TXN100001	Furniture	Chair	Chair	37103.37	37103.37	4	148413.48	Completed	Central	High	2025-08-27	UPI	Julie Miller
8527	P80284	TXN100004	Groceries	Sugar	Sugar	13299.59	13299.59	1	13299.59	Completed	West	High	2025-07-19	Debit Card	Erica Stanley
4733	P47930	TXN100005	Automotive	Car Cover	Car Cover	35258.13	35258.13	1	35258.13	Completed	South	High	2025-01-24	Debit Card	Elizabeth Nelson
1916	P40021	TXN100007	Sports	Yoga Mat	Yoga Mat	11058.8	11058.8	6	66352.8	Completed	East	Medium	2024-10-01	Cash	Nicholas Thomas
9348	P74686	TXN100009	Automotive	Bike Gloves	Bike Gloves	18163.62	18163.62	3	54490.86	Completed	North	High	2024-06-05	Wallet	Kevin Marsl
3591	P69470	TXN100011	Sports	Basketball	Basketball	5666.45	5666.45	7	39665.15	Completed	East	Low	2024-07-01	Net Banking	Larry Lewis
3646	P80697	TXN100012	Toys	Remote Car	Remote Car	10025.66	10025.66	6	60153.96	Completed	Central	Medium	2025-05-15	Cash	NULL
2049	P54313	TXN100015	Toys	Doll	Doll	6138.11	6138.11	4	24552.44	Completed	Central	NULL	2024-08-23	Net Banking	Heather Hous
7669	P34957	TXN100017	Automotive	Tyre Inflator	Tyre Inflator	23686.28	23686.28	8	189490.24	Completed	North	Low	2025-03-17	Debit Card	Andre Lev
5033	P85880	TXN100023	Books	Data Science	Data Science	6082.34	6082.34	10	60823.4	Completed	Central	High	2025-03-05	Wallet	Jennifer Montgomery
5956	P90173	TXN100026	Groceries	Bread	Bread	7971.57	7971.57	9	71744.13	Completed	Central	Medium	2025-11-10	Cash	Daryl Cooper
1158	P24663	TXN100029	Clothing	Shirt	Shirt	27628.85	27628.85	7	193401.95	Completed	South	Low	2024-02-06	Debit Card	John Perez
6378	P46517	TXN100033	Books	Python Guide	Python Guide	33035.66	33035.66	7	231249.62	Completed	East	Medium	2025-10-18	Net Banking	Robert Green
2669	P40529	TXN100052	Beauty	Shampoo	Shampoo	3892.22	3892.22	4	15568.88	Completed	Central	High	2024-12-21	Net Banking	Samantha Garz
7882	P74251	TXN100053	Furniture	Wardrobe	Wardrobe	32412.41	32412.41	1	32412.41	Completed	West	Medium	2025-10-12	UPI	NULL
4155	P51359	TXN100059	Sports	Basketball	Basketball	12645.04	12645.04	2	25290.08	Completed	Central	Low	2024-07-08	Credit Card	Frank William
5136	P72616	TXN100062	Home Decor	Lamp	Lamp	9299.75	9299.75	1	9299.75	Completed	North	Medium	2025-05-18	Cash	Jamie Wheeler
2395	P42530	TXN100063	Clothing	Jeans	Jeans	40575.99	40575.99	5	202879.95	Completed	Central	Medium	2025-09-12	Net Banking	James Burns
5352	P28643	TXN100067	Beauty	Face Wash	Face Wash	27273.66	27273.66	3	81820.98	Completed	North	High	2024-11-10	Net Banking	Terry Morton
7561	P82102	TXN100071	Groceries	Eggs	Eggs	34714.32	34714.32	5	173571.6	Completed	West	High	2024-11-18	Debit Card	Miss Kara Carter M

only showing top 20 rows

Q9. Explain Predicate Pushdown.

Answer

Predicate Pushdown is an optimization technique used by Spark when reading Parquet files. Instead of loading the entire dataset into memory, Spark pushes filter conditions directly to the Parquet reader.

Benefits include:

- Reads only the required row groups.
- Skips irrelevant data blocks.
- Reduces disk I/O.
- Decreases memory usage.
- Improves query execution speed.

This optimization makes Parquet highly efficient for analytical workloads.

Q10. Add final_price column.

Explanation

A new column named **final_price** was created by multiplying the **base_price** column by **1.18** to include an 18% tax.

Code Screenshot

PySpark Code

```
from pyspark.sql.functions import col

df = df.withColumn(
    "final_price",
    col("base_price") * 1.18
)

df.show(5)
```

✓ 0.1s

Output Screenshot

user_id	product_id	transaction_id	category	product_name	new_name	price	base_price	quantity	amount	status	region	priority	order_date	payment_method	customer_name	c:
4657	P28289	TXN1000001	Furniture	Chair	Chair	37103.37	37103.37	4	148413.48	Completed	Central	High	2025-08-27	UPI	Julie Miller	De
1434	P83563	TXN1000002	Furniture	Table	Table	11709.78	11709.78	10	117097.8	Pending	Central	Medium	2024-08-13	UPI	William Torres	Kolk
5552	P30379	TXN1000003	Electronics	Smartphone	Smartphone	34937.16	34937.16	6	209622.96	Pending	East	High	2024-04-04	UPI	Sarah Byrd	Mum
8527	P80284	TXN1000004	Groceries	Sugar	Sugar	13299.59	13299.59	1	13299.59	Completed	West	High	2025-07-19	Debit Card	Erica Stanley	Kolk
4733	P47930	TXN1000005	Automotive	Car Cover	Car Cover	35258.13	35258.13	1	35258.13	Completed	South	High	2025-01-24	Debit Card	Elizabeth Nelson	Luck

only showing top 5 rows

Q11. Difference between Transformations and Actions.

Answer

Transformations create a new DataFrame but do not execute immediately. They are evaluated lazily and become part of the execution plan.

Examples of Transformations

- filter()
- select()

Actions trigger the execution of all pending transformations and produce a result.

Examples of Actions

- show()
- count()

Transformations	Actions
Lazy execution	Immediate execution
Return a new DataFrame	Return a result
Build the DAG	Execute the DAG

Q12. Load a Parquet file, remove rows where user_id is null, and save the result as CSV.

Explanation

A Parquet dataset was read into a Spark DataFrame, rows containing null values in the **user_id** column were filtered out, and the cleaned dataset was written as a CSV file. *(If you encountered the Windows Hadoop issue, mention that the implementation follows the required Spark syntax even though execution was affected by local environment configuration.)*

Code Screenshot

```
PySpark Code

spark.read.parquet("path/to/input") \
    .filter(col("user_id").isNull()) \
    .write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("path/to/output")
```

Q13. Difference between Client Mode and Cluster Mode.

Answer	
Client Mode	Cluster Mode
Driver runs on the local machine.	Driver runs inside the cluster.
Suitable for development and testing.	Suitable for production environments.
Application depends on the client machine.	Application continues even if the client disconnects.
Easier for debugging.	Better fault tolerance and scalability.

Cluster Mode is generally preferred for production workloads because it provides better reliability and resource management.

Q14. Filter region = North OR priority = High.

Explanation

The DataFrame was filtered using the OR (|) operator to retrieve records where the **region** is North or the **priority** is High.

Code Screenshot

```
PySpark Code

from pyspark.sql.functions import col

df.filter(
    (col("region") == "North") |
    (col("priority") == "High")
).show()
```

4] ✓ 0.1s

Output Screenshot

user_id	product_id	transaction_id	category	product_name	new_name	price	base_price	quantity	amount	status	region	priority	order_date	payment_method	customer_name
4657	P28289	TXN100001	Furniture	Chair	Chair	37103.37	37103.37	4	148413.48	Completed	Central	High	2025-08-27	UPI	Julie Miller
5552	P30379	TXN100003	Electronics	Smartphone	Smartphone	34937.16	34937.16	6	209622.96	Pending	East	High	2024-04-04	UPI	Sarah By
8527	P80284	TXN100004	Groceries	Sugar	Sugar	13299.59	13299.59	1	13299.59	Completed	West	High	2025-07-19	Debit Card	Erica Stanl
4733	P47930	TXN100005	Automotive	Car Cover	Car Cover	35258.13	35258.13	1	35258.13	Completed	South	High	2025-01-24	Debit Card	Elizabeth Nels
9348	P74686	TXN100009	Automotive	Bike Gloves	Bike Gloves	18163.62	18163.62	3	54490.86	Completed	North	High	2024-06-05	Wallet	Kevin Mar
8668	P79352	TXN100010	Automotive	Helmet	Helmet	19299.89	19299.89	10	192998.9	Cancelled	Central	High	2025-11-27	Wallet	Connie Mum
2290	P21226	TXN100013	Sports	Cricket Bat	Cricket Bat	2990.47	2990.47	10	29904.7	Returned	North	Low	2024-05-08	Credit Card	Gregory Farl
7669	P34957	TXN100017	Automotive	Tyre Inflator	Tyre Inflator	23686.28	23686.28	8	189490.24	Completed	North	Low	2025-03-17	Debit Card	Andre Li
1993	P62772	TXN100018	Home Decor	Wall Clock	Wall Clock	33700.89	33700.89	2	67401.78	Cancelled	North	High	2024-07-15	Credit Card	Mark Clai
5092	P19880	TXN100019	Clothing	Shirt	Shirt	9256.05	9256.05	8	74048.4	Returned	Central	High	2024-02-21	Wallet	Laura Lop
8886	P38016	TXN100020	Furniture	Table	Table	8398.92	8398.92	8	67191.36	Returned	North	High	2025-01-23	Cash	Ashley Pier
8973	P30289	TXN100021	Home Decor	Lamp	Lamp	21208.07	21208.07	9	190872.63	Pending	East	High	2024-02-29	Net Banking	Dr. Jill Wel
9238	P79615	TXN100022	Groceries	Rice	Rice	2602.1	2602.1	8	20816.8	Pending	North	Low	2024-03-23	Credit Card	Anthony Wrig
5033	P85880	TXN100023	Books	Data Science	Data Science	6082.34	6082.34	10	60823.4	Completed	Central	High	2025-03-05	Wallet	Jennifer Montgome
5462	P46930	TXN100027	Furniture	Chair	Chair	37146.59	37146.59	3	111439.77	Pending	East	High	2025-12-04	Wallet	Howard Pier
2512	P93136	TXN100028	Home Decor	Lamp	Lamp	45276.29	45276.29	1	45276.29	Returned	East	High	2024-01-04	Debit Card	Luis Flower
4878	P31299	TXN100031	Toys	Doll	Doll	48694.47	48694.47	3	146083.41	Pending	West	High	2024-07-02	Wallet	Mark Blau
7267	P15075	TXN100032	Books	SQL Basics	SQL Basics	8043.97	8043.97	2	16087.94	Returned	South	High	2025-04-16	Debit Card	Isabella Martin
3925	P86099	TXN100034	Groceries	Rice	Rice	5855.55	5855.55	5	29277.75	Cancelled	North	High	2025-09-02	UPI	Alicia Ram
7311	P85574	TXN100035	Beauty	Body Lotion	Body Lotion	49213.37	49213.37	2	98426.74	Pending	East	High	2025-12-26	UPI	Lindsey Nguy

only showing top 20 rows

Q15. Why is `.show(5)` safer than `.collect()`?

Answer

The `.show(5)` method retrieves and displays only the first five rows of a DataFrame, making it safe for exploring very large datasets.

In contrast, `.collect()` retrieves **all rows** from the distributed dataset and transfers them to the Driver's memory.

Using `.collect()` on a multi-terabyte dataset can:

- Cause Out of Memory (OOM) errors.
- Crash the Driver application.
- Increase network traffic and execution time.
- Reduce overall application performance.

Therefore, `.show(5)` is recommended for inspecting large datasets because it minimizes memory usage and avoids unnecessary data transfer.

Conclusion

This assignment demonstrated the practical implementation of Apache Spark architecture and PySpark DataFrame operations for distributed data processing. Various concepts such as Driver, Cluster Manager, Executors, Lazy Evaluation, DAG-based fault tolerance, Predicate Pushdown, schema handling, filtering, transformations, actions, and data type conversion were successfully explored. A custom Retail Sales dataset was generated and processed using Spark, highlighting the efficiency of Spark's distributed computing model and its suitability for handling large-scale data processing tasks.

References

- [Apache Spark Documentation](#)
- [PySpark API Documentation](#)
- Celebal Technologies Data Engineering Internship Material